CSE 144 Final Project Report

100-Class Few-Shot Image Classification via CLIP Feature Extraction

UCSC CSE 144 Applied Machine Learning - Spring 2026

Team: Ruoyu Hu (2046446), Hyerin Chung (2256365), Jacob Diaz

1. Introduction

The central challenge of this project was building a 100-class image classifier with only 10 labeled examples per class, a classic few-shot learning scenario. With just approximately 1,000 training images total, conventional approaches such as training a convolutional neural network from scratch fail almost immediately due to severe overfitting. Even fine-tuning a large pretrained model is problematic at this scale, since there is simply not enough labeled data to meaningfully update millions of parameters without the model memorizing the training set rather than generalizing.

Our approach reframed the problem from a fine-tuning challenge into a representation learning question: if a pretrained model already embeds images into a feature space that separates our 100 classes well, a simple linear classifier trained on top of those frozen features should still achieve strong results. This insight led us to OpenAI's CLIP (Contrastive Language-Image Pretraining), a vision-language model pretrained on 400 million image-text pairs. CLIP's visual features turned out to transfer remarkably well to this dataset, ultimately achieving 88.18% test accuracy compared to a 16.4% baseline from standard fine-tuning.

In summary, our approach proceeds in three stages, (1) extract frozen CLIP features from all training and test images, (2) train a Logistic Regression classifier on those features, and (3) ensemble two CLIP backbone sizes to combine complementary representations. The entire pipeline runs in under five minutes on CPU and requires no GPU.

2. Dataset

The dataset consists of 100 classes sampled from diverse image sources, with exactly 10 training images per class, for a total of 1,000 training images. The test set contains 1,000 unlabeled images. All images are in color (RGB) but may vary in spatial resolution, so we resize all inputs to 224x224 pixels before feature extraction.

The directory structure follows a straightforward layout. The train/ directory contains 100 subdirectories named "0" through "99", each holding approximately 10 JPEG images (0.jpg through 9.jpg). The test/ directory contains 1,000 unlabeled images named 0.jpg through 999.jpg, along with a sample_submission.csv template.

Label ordering is critical for correct evaluation. We use the integer folder names directly as class labels: folder "0" maps to label 0, folder "1" maps to label 1, and so on through folder "99" to label 99. We verified this mapping explicitly in our data loading code by sorting directory names numerically rather than lexicographically. Scrambled label ordering would reduce best-case performance to random chance.

For CLIP feature extraction, preprocessing follows CLIP's built-in pipeline: images are resized and center-cropped to 224x224, then normalized using CLIP's published mean and standard deviation values. No additional augmentation is applied at this stage since CLIP's weights are frozen and augmentation would only increase inference time without changing the extracted features. For the baseline ResNet50 fine-tuning experiments, we applied standard training augmentations including RandomResizedCrop(224), RandomHorizontalFlip, and ColorJitter

with brightness, contrast, and saturation each varied by plus or minus 0.2. Validation images used a deterministic Resize(256) followed by CenterCrop(224).

3. Implementation

3.1 Model

3.1.1 Pretrained Backbone

Our final model uses OpenAI's CLIP ViT-L/14 and ViT-B/16 as frozen feature extractors. CLIP was chosen for two reasons. First, it is pretrained on 400 million image-text pairs scraped from the internet, which means its visual representations are optimized to align with open-ended natural language descriptions rather than to discriminate among a fixed set of ImageNet categories. This produces a rich, semantically structured embedding space that generalizes well to novel classes. Second, because we only freeze and extract from CLIP rather than fine-tuning it, the entire training regime collapses to fitting a linear classifier, which is extremely data-efficient and avoids overfitting on 10 examples per class.

We also benchmarked ResNet18 and ResNet50 as fine-tuning baselines. ResNet models are pretrained on ImageNet and represent the conventional transfer learning approach. As detailed in the experiments section, they performed poorly under this data regime.

3.1.2 Architecture and Classifier Head

For each image, we pass it through CLIP's vision encoder under torch.no_grad() and extract the output embedding. ViT-B/16 produces a 512-dimensional embedding, and ViT-L/14 produces a 768-dimensional embedding. For our ensemble model, we L2-normalize each embedding independently and then concatenate them into a single 1,280-dimensional feature vector. This concatenated vector is then fed to a single Logistic Regression classifier with 100 output classes.

We chose Logistic Regression over a small neural network or SVM for three practical reasons. The optimization is convex, guaranteeing a globally optimal solution with no sensitivity to initialization. L2 regularization naturally penalizes overfitting. And the lbfgs solver is fully deterministic given a fixed random seed, making results exactly reproducible. We tuned the regularization strength C on a held-out validation split and found C=10 worked best, indicating the classifier benefited from relatively weak regularization given the quality of the CLIP features.

3.1.3 Fine-Tuning Strategy

Because our final model uses frozen CLIP features, there is no fine-tuning of the backbone. All gradient updates are restricted to the Logistic Regression classifier, which has only 1,280 x 100 parameters. This is intentional: the few-shot regime (10 examples per class) makes fine-tuning a 307-million-parameter model such as ViT-L/14 extremely risky, as the model would overfit the training set before learning any generalizable patterns.

For the ResNet50 baseline experiments, we applied a differential learning rate strategy: a rate of 1e-4 for the pretrained backbone and 1e-3 for the newly added classification head. This helps preserve the pretrained representations while allowing the head to adapt more rapidly.

3.2 Training

For the final CLIP-based classifier, we trained a Logistic Regression model using scikit-learn on top of the extracted CLIP features. Since the task is a 100-class classification problem, the classifier internally optimizes a multi-class cross-entropy objective. We used the lbfgs solver with max_iter = 3000, C = 10.0, and random_state = 42. The regularization parameter C was selected based on validation performance, with C = 10.0 giving the best result among the values we tested.

Unlike a standard PyTorch training setup, the final CLIP pipeline did not require a manually defined learning rate, scheduler, batch size, epoch count, or weight decay. Feature extraction was performed once using the pretrained CLIP encoders, and the resulting feature matrix was used directly to fit the Logistic Regression classifier. This made the final training process lightweight and deterministic.

For the ResNet50 baseline experiments, we used a conventional supervised fine-tuning setup. The model was trained with CrossEntropyLoss and the AdamW optimizer. We used separate learning rates for different parts of the model, 1e-4 for the pretrained backbone and 1e-3 for the final classification layer. The batch size was 32, the maximum number of epochs was 30, and weight decay was set to 1e-4. We also used a CosineAnnealingLR scheduler and early stopping with a patience of 5 epochs based on validation accuracy.

All experiments were implemented in Python 3.10 using PyTorch 2.1.0, torchvision 0.16.0, OpenAI CLIP 1.0.1, scikit-learn 1.3.0, NumPy 1.24.3, and Pillow 9.5.0. OpenAI CLIP was installed using ‘pip install git+https://github.com/openai/CLIP.git’. The ResNet50 baseline used CUDA when available, while the final CLIP feature extraction pipeline was run on CPU by setting device = "cpu".

4. Experiments

4.1 Baseline Setup

Our initial baseline was a ResNet18 pretrained on ImageNet, with its final fully connected layer replaced by a new linear layer mapping to 100 classes. We fine-tuned for 30 epochs using AdamW with a learning rate of 1e-3 and no learning rate scheduling. This achieved 16.4% test accuracy, roughly four times random chance (1%) but far below the 60% required passing threshold. The low accuracy reflects the fundamental difficulty of fine-tuning a multi-million-parameter network on 1,000 samples.

We then set up a more careful fine-tuning baseline using ResNet50 with differential learning rates (1e-4 for the backbone, 1e-3 for the head), AdamW optimization with weight decay of 1e-4, cosine learning rate annealing over 30 epochs, and early stopping with patience of 5. Validation accuracy peaked at 66.7% at epoch 11 before early stopping triggered at epoch 16, well below the 88.2% achieved by frozen CLIP features. While ResNet50 fine-tuning does pass the 60% baseline threshold, the gap confirms that frozen CLIP representations are significantly more effective for this few-shot regime.

4.2 Hyperparameter Tuning and Validation Method

For the CLIP Logistic Regression pipeline, we created a held-out validation split of 10% of the training data (100 images) using a fixed random seed of 42, with the remaining 900 images used for fitting the classifier. We selected C=10 based on empirical testing across several values, finding that relatively weak regularization worked best given the high quality of the CLIP features.

For the ResNet fine-tuning experiments, we similarly held out 10% of training data and monitored validation accuracy every epoch, using early stopping with patience of 5 to prevent overfitting.

4.3 Ablations

We ran a systematic ablation across CLIP model sizes to understand the effect of backbone capacity on downstream accuracy. Results are reported on the Kaggle public leaderboard:

Model	Kaggle Accuracy
ResNet18 (fine-tuned)	16.4%
ResNet50 (fine-tuned)	66.7%
CLIP ViT-B/32	69.1%
CLIP ViT-B/16	80.0%
CLIP ViT-L/14	87.3%
Ensemble: ViT-B/16 + ViT-L/14	88.2%

Larger CLIP backbones consistently outperform smaller ones, reflecting that higher-capacity vision transformers produce more discriminative feature representations. The jump from ViT-B/32 to ViT-B/16 (69.1% to 80.0%) is attributable to finer patch resolution: ViT-B/16 divides each image into 14x14 patches of 16 pixels, capturing more local texture and detail. ViT-L/14 further increases capacity with a larger model dimension.

We also ablated data augmentation during fine-tuning (adding or removing ColorJitter and RandomHorizontalFlip for the ResNet experiments) and found modest effects, consistent with the conclusion that data quantity, not augmentation, is the limiting factor at this scale.

5. Results

Our final ensemble model achieves 88.18% test accuracy on the Kaggle leaderboard, placing us 12th out of all submissions. Training accuracy on the 900-sample training split is 99.81%, so the Logistic Regression fits the training data nearly perfectly. The approximately 12-point gap between training and test accuracy is expected given that only 10 examples per class are available; it reflects the irreducible difficulty of generalizing from such few samples rather than overfitting in the conventional sense.

The most striking result is the gap between fine-tuning and frozen CLIP extraction. ResNet18 fine-tuned achieves 16.4%; frozen CLIP ViT-L/14 achieves 87.3%. This 70-point difference is not primarily attributable to model size. ResNet50 has approximately 25 million parameters and ViT-L/14 has 307 million, but a similar-capacity comparison using CLIP ViT-B/32 still yields 69.1% versus ResNet18's 16.4%. The key distinction is what the features represent. ImageNet-pretrained features are optimized to discriminate among 1,000 fixed categories. CLIP features are optimized to align visual representations with open-ended natural language, producing a semantically richer embedding space that generalizes far better to unseen classes.

The ensemble gain from combining ViT-B/16 and ViT-L/14 (87.3% to 88.2%) was small but consistent across multiple Kaggle submissions. The two models use different patch resolutions (16 pixels vs 14 pixels) and different embedding dimensions (512 vs 768), so they tend to make different errors. Concatenating both normalized embeddings and fitting a single Logistic Regression lets the classifier exploit both views simultaneously.

We did not produce training/validation loss curves for the CLIP pipeline because there is no iterative training loop: feature extraction is a single forward pass per image, and Logistic Regression fitting converges in one call to the lbfgs solver. For the ResNet baseline experiments, validation accuracy oscillated significantly across epochs, never producing a stable convergence curve, which is itself diagnostic of the fundamental mismatch between model capacity and data quantity.

6. Discussion

6.1 What Worked Best and Why

The single most impactful decision was switching from fine-tuning to frozen CLIP feature extraction. Within the CLIP pipeline, increasing backbone size produced consistent gains, and the ensemble of ViT-B/16 and ViT-L/14 provided a small additional boost by combining complementary views. The choice of Logistic Regression over alternative classifiers (SVM, small MLP) was validated empirically and justified theoretically: convexity guarantees the optimal solution, and L2 regularization smoothly handles the under-constrained regime where features outnumber training samples.

Fundamentally, the success of CLIP in this setting comes down to training objective alignment. A model trained to match images to natural language descriptions develops representations organized around semantic concepts rather than pixel statistics. This transfers naturally to a diverse 100-class dataset where the classes span many domains.

6.2 Failure Cases and Observations

Because we do not have ground truth labels for the test set, we cannot directly identify which test images were misclassified. Based on visual inspection of the training classes, we expect errors to cluster around visually similar categories where distinguishing features are fine-grained or contextual rather than coarse. Animal species with similar fur patterns or food categories with overlapping colors and textures are likely sources of confusion. This is a known limitation of frozen CLIP features, which are broadly semantic but not specifically optimized for fine-grained discrimination.

The 12-point training-to-test gap is worth noting. With only 10 training examples per class, the Logistic Regression fits the training embeddings almost perfectly (99.81%), but the true class boundaries in CLIP's feature space are estimated from very few points. Some test images fall into ambiguous regions of the feature space that the classifier cannot confidently resolve.

6.3 Limitations and Next Steps

The primary limitation of our approach is that it relies entirely on the quality of CLIP's pretrained representations. For classes that are very different from anything in CLIP's pretraining distribution, frozen feature extraction may underperform. A natural next step would be few-shot fine-tuning methods such as prompt tuning or adapter layers, which update only a small number of parameters on top of the frozen backbone. These approaches would allow domain-specific adaptation without the overfitting risk of full fine-tuning.

Another direction is test-time augmentation: extracting features from multiple augmented views of each test image and averaging the predictions. This is straightforward to implement and could provide an additional accuracy boost without any change to the training procedure. We did not pursue this given that we had already surpassed the project's target accuracy threshold.

7. Reproducibility

All experiments use a fixed random seed of 42 throughout. CLIP feature extraction is fully deterministic: all inference runs under torch.no_grad() and CLIP applies no stochastic operations. The Logistic Regression solver (lbfgs) is also deterministic given a fixed seed. Running the training script from scratch on the same dataset will produce an identical clip_classifier.pkl and identical predictions.

Package versions used in our environment:

- Python 3.10
- PyTorch 2.1.0
- torchvision 0.16.0
- openai-clip 1.0.1 (via pip install git+https://github.com/openai/CLIP.git)
- scikit-learn 1.3.0
- numpy 1.24.3
- Pillow 9.5.0

To reproduce training and generate submission.csv, run the following two commands in order:

- python train.py
- python predict.py

Feature extraction takes approximately 3 to 4 minutes on CPU. Logistic Regression fitting takes under 30 seconds. Total runtime from raw images to submission.csv is under 5 minutes on a standard laptop CPU.

8. Team Contributions

Ruoyu Hu: Led model architecture design and the full training pipeline. This included selecting CLIP as the primary backbone after the ResNet fine-tuning baseline failed to converge, implementing frozen feature extraction for both ViT-B/16 and ViT-L/14, applying L2 normalization and constructing the concatenated 1280-dim ensemble feature vector, and fitting and tuning the Logistic Regression classifier. Ran the full ablation study across all CLIP backbone sizes to determine the optimal configuration. Wrote Sections 1, 3, 4, 5, and 7 of this report.

Hyerin Chung: Led all data preprocessing and augmentation work. This included setting up the data loading pipeline with correct numerical label ordering (folder "0" to label 0 through folder "99" to label 99), implementing CLIP's built-in preprocessing pipeline for feature extraction, and designing the ResNet50 fine-tuning augmentation strategy using RandomResizedCrop, RandomHorizontalFlip, and ColorJitter. Also managed the Kaggle submission pipeline and verified that predictions matched the expected CSV format. Wrote Sections 2, 6, and 8 of this report.

Jacob Diaz: Led hyperparameter tuning and model evaluation. This included setting up the held-out validation split, testing regularization strength values for the Logistic Regression classifier, and selecting C=10 as the best-performing configuration. Also handled evaluation metric tracking across experiments, verified Kaggle submission scores against validation results, and contributed to the ResNet50 fine-tuning baseline experiments including learning rate and scheduler configuration. Assisted with writing Sections 4 and 5 of this report.

9. References

Radford, A., Kim, J. W., Hallacy, C., Ramesh, A., Goh, G., Agarwal, S., Sastry, G., Askell, A., Mishkin, P., Clark, J., Krueger, G., & Sutskever, I. (2021). Learning transferable visual models from natural language supervision. In International Conference on Machine Learning (pp. 8748-8763). PMLR.

He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep residual learning for image recognition. In Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (pp. 770-778).

TorchVision Contributors. (2023). TorchVision pretrained models. PyTorch Documentation. https://pytorch.org/vision/stable/models.html

OpenAI. (2021). CLIP: Connecting text and images. https://openai.com/research/clip